

.....



Análise e Desenvolvimento de Sistemas

Estrutura de Dados

Prof. Veríssimo

Atividade Final: Consolidação do Conteúdo Aplicado na Disciplina

carlos.pereira01@cps.sp.gov.br

Maio/2026

Preâmbulo Técnico e Justificativa de Competências

O presente documento constitui a avaliação consolidada da disciplina de **Estrutura de Dados**. No âmbito do desenvolvimento de sistemas de alta performance, o domínio pleno dos fundamentos de baixo nível e das estruturas lógicas avançadas não representa um mero diferencial acadêmico, mas sim um requisito técnico indispensável para a construção de soluções robustas, escaláveis e otimizadas.

A consolidação de competências nesta etapa exige que o tecnólogo em Análise e Desenvolvimento de Sistemas detenha a compreensão exaustiva e a capacidade de aplicação prática dos seguintes pilares fundamentais abordados:

- **Ponteiros e Manipulação Direta de Memória:** Compreensão cirúrgica dos operadores de referência (&) e desreferenciação (*), aritmética aplicada a ponteiros e sua equivalência estrutural com vetores, além da passagem de parâmetros por referência em funções para otimização do ciclo de vida das variáveis.
- **Recursividade:** Capacidade de abstração matemática e lógica para segmentação de problemas complexos em subproblemas auto-similares, identificação precisa de casos base para prevenção de *stack overflow* e gerenciamento do fluxo de execução na pilha de chamadas (*stack*).
- **Árvores Binárias de Busca (ABB):** Domínio das propriedades estruturais e terminologias de grafos acíclicos direcionados, englobando a correta determinação de graus de nós, identificação de folhas, mapeamento de ancestrais e descendentes, e o cálculo exato de alturas e profundidades no arranjo hierárquico.

Para este fim, o documento estrutura-se em desafios práticos rigorosos que transcendem a simples reprodução sintática de códigos em linguagem C. Cada seção foi projetada para **estimular o seu raciocínio analítico**, exigindo que você valide hipóteses, analise a complexidade assintótica dos algoritmos e correlacione o comportamento das variáveis com a alocação real na memória física. Através desta abordagem integrada, consolida-se a transição da programação puramente empírica para a verdadeira engenharia de software aplicada.

Disciplina: Estrutura de Dados

Atividade Avaliativa N2-3

Consolidação de Competências em Linguagem C

Apresentação da Atividade

Programar em C é dialogar diretamente com a máquina — e nenhum recurso torna esse diálogo mais poderoso (e mais perigoso) do que o **ponteiro**. Dominar ponteiros significa compreender *onde* os dados vivem na memória e *como* manipulá-los com precisão cirúrgica. Aliado a isso, a **recursividade** reveals uma forma elegante de decompor problemas complexos em soluções simples e auto-similares — princípio que está na base de estruturas como a **árvore binária**, onipresente em bancos de dados, sistemas de arquivos e compiladores.

Esta atividade é uma oportunidade de **verificar, consolidar e aprofundar** sua compreensão desses três pilares da disciplina. Cada exercício foi cuidadosamente elaborado para ir além da mera reprodução de código: você será desafiado a *raciocinar, depurar e justificar* suas escolhas de implementação.

Tópicos abordados:

1. Ponteiros: operadores de referência e desreferenciação; aritmética de ponteiros; ponteiro com *arrays*; passagem de ponteiro em funções.
2. Recursividade.
3. Árvore binária: identificação de elementos de um nó (altura, profundidade, ancestrais, descendentes e grau do nó).

1. Ponteiros em C

1.1. Conceitos fundamentais

Referência rápida — Operadores de Ponteiro

- `&var` — operador de **referência**: retorna o *endereço de memória* da variável `var`.
- `*ptr` — operador de **desreferenciação**: acessa o *valor armazenado* no endereço apontado por `ptr`.
- **Aritmética de ponteiros**: somar ou subtrair um inteiro n a um ponteiro avança ou recua $n \times \text{sizeof}(\text{tipo})$ bytes na memória.
- **Ponteiro e array**: o nome de um array é, em essência, um ponteiro para seu primeiro elemento — $v[i] \equiv *(v + i)$.

Exercício 01 — Operadores de Referência, Desreferenciação e Aritmética de Ponteiros

Contexto: Você precisa escrever uma função que *troca os valores* de duas variáveis inteiras sem utilizar uma variável temporária adicional — apenas ponteiros.

Parte A — Troca com ponteiros (operadores & e *)

Implemente a função:

```
1 void trocar(int *a, int *b);
```

A função deve trocar os valores apontados por **a** e **b** usando **aritmética de inteiros** (operações de soma e subtração) — *sem* variável temporária e *sem* XOR. No **main**, declare duas variáveis, exiba seus endereços com `%p` e seus valores **antes** e **depois** da troca.

Parte B — Aritmética de ponteiros in um vetor

Dado o vetor:

```
1 int v[] = {10, 20, 30, 40, 50};
```

Utilizando **apenas aritmética de ponteiros** (proibido o uso do operador `[ ]`), percorra o vetor e:

- Imprima cada elemento e seu endereço.
- Calcule e imprima a soma de todos os elementos.
- Inverta o vetor *in-place* (sem criar outro vetor) usando dois ponteiros: um iniciando no começo e outro no final.

Requisitos técnicos:

- Nenhum acesso por índice (`v[i]`) é permitido na Parte B.
- Use `printf` para exibir endereços no formato `0x...`
- Comente cada linha que utilize `&` ou `*` explicando o que ocorre.

Exercício 02 — Ponteiro com Array e Passagem de Ponteiro em Funções

Contexto: Você deve implementar um *mini-sistema de estatísticas* que opera sobre um vetor de números reais, manipulando-o exclusivamente por meio de ponteiros passados a funções.

Implemente as seguintes funções, todas recebendo um ponteiro para o início do vetor e o tamanho do mesmo:

```
1  /* Calcula e RETORNA a media aritmetica */
2  float calcularMedia(float *vet, int n);
3
4  /* Encontra o maior e o menor valor;
5     armazena nos ponteiros recebidos */
6  void encontrarExtremos(float *vet, int n,
7                        float *maior, float *menor);
8
9  /* Normaliza o vetor: cada elemento eh
10     dividido pelo valor maximo */
11 void normalizar(float *vet, int n);
12
13 /* Imprime o vetor formatado */
14 void imprimirVetor(float *vet, int n, const char *titulo)
    ;
```

No main:

1. Declare `float notas[] = {7.5f, 3.2f, 9.8f, 6.0f, 5.5f, 8.1f};`
2. Chame `imprimirVetor` antes e depois de `normalizar`.
3. Exiba a média, o maior e o menor valor.
4. Demonstre que a função **normalizar** **alterou** o vetor original (não uma cópia), discutindo brevemente em comentário no código por que isso ocorre com a passagem por ponteiro.

Requisitos técnicos:

- Toda iteração dentro das funções deve usar incremento de ponteiro (`ptr++`) — sem índices.
- `maior` e `menor` são ponteiros recebidos por parâmetro; a função *não* os retorna — escreve diretamente no endereço indicado.
- O arquivo deve compilar sem avisos com `-Wall -Wextra`.

2. Recursividade

Referência rápida — Anatomia de uma Função Recursiva

- **Caso base:** condição que *encerra* a recursão — sem ele, a pilha estoura (*stack overflow*).
- **Caso recursivo:** chamada da própria função com um argumento *mais próximo* do caso base.
- Cada chamada empilha um *stack frame* com suas variáveis locais; a pilha desempilha na ordem inversa ao retornar.

Exercício 03 — Torres de Hanói Recursivo com Rastreamento

Contexto: O problema das Torres de Hanói é o exemplo canônico de recursividade — simples de enunciar, impossível de resolver eficientemente de forma iterativa sem simular a pilha manualmente.

Parte A — Implementação

Implemente a função recursiva:

```
1 void hanoi(int n, char origem, char destino, char  
    auxiliar);
```

que imprime cada movimento necessário para transferir n discos da haste `origem` para a haste `destino` usando `auxiliar`.

Parte B — Análise de complexidade

Adicione um contador global que contabiliza o **número de movimentos** realizados para cada valor de n (de 1 a 6). Preencha, em comentário no código, a tabela:

n	Movimentos (contado)	$2^n - 1$ (esperado)
1		
2		
3		
4		
5		
6		

Parte C — Profundidade da recursão

Adicione um parâmetro `int nivel` à função e imprima a profundidade atual antes de cada movimento, indentando a saída com espaços proporcionais (`nivel * 2` espaços).

Requisitos técnicos:

- A solução deve ser *puramente recursiva* — nenhuma estrutura de pilha auxiliar manual é permitida.
- Teste com $n = 1$, $n = 3$ e $n = 4$.

Exercício 04 — Busca Binária Recursiva e Potenciação Rápida

Contexto: Dois algoritmos clássicos que demonstram como a recursão reduz a complexidade de problemas ao *dividir* o espaço de busca ou de cálculo pela metade a cada chamada.

Parte A — Busca Binária Recursiva

Implemente:

```
1  /* Retorna o INDICE do elemento se encontrado,
2     ou -1 caso contrario.
3     O vetor deve estar ordenado em ordem crescente. */
4  int buscaBinaria(int *vet, int inicio, int fim, int alvo)
    ;
```

No main, declare `int dados[] = {2, 5, 8, 12, 16, 23, 38, 45, 72, 91}`; e busque ao menos três valores: um presente, um ausente e o primeiro elemento. Para cada busca, imprima o resultado e **quantas chamadas recursivas** foram necessárias (use uma variável estática ou parâmetro de contagem).

Parte B — Potenciação Rápida (*Fast Exponentiation*)

Implemente:

```
1  /* Calcula base^exp de forma recursiva eficiente.
2     Complexidade: O(log exp)
3                                     */
4  long long potenciaRapida(long long base, int exp);
```

Dica: se exp for par, use $b^{\text{exp}} = (b^{\text{exp}/2})^2$; se ímpar, use $b^{\text{exp}} = b \cdot b^{\text{exp}-1}$.
Teste com: 2^{10} , 3^7 , 5^0 , 7^{12} . Compare o número de chamadas recursivas com a abordagem ingênua ($b^n = b \cdot b^{n-1}$) — mostre a diferença em comentário.

Requisitos técnicos:

- Nenhuma função de biblioteca (`pow`, `math.h`) pode ser usada.
- Identifique e documente claramente o caso base e o caso recursivo de cada função.

3. Árvore Binária — Propriedades dos Nós

3.1. Terminologia essencial

Elementos de um Nó em uma Árvore Binária

```
graph TD; A((A)) --- B((B)); A --- C((C)); B --- D((D)); B --- E((E)); D --- H((H)); D --- I((I)); C --- F((F)); C --- G((G)); F --- J((J));
```

Conceito	Definição
Raiz	Nó sem pai — aqui: A
Grau do nó	Número de filhos diretos — A tem grau 2; H tem grau 0
Folha	Nó com grau 0 (sem filhos) — E, G, H, I, J
Profundidade	Distância (em arestas) do nó até a raiz — A: prof. 0; D: prof. 2
Altura do nó	Maior caminho do nó até uma folha descendente — A: alt. 3; D: alt. 1
Altura da árvore	Altura da raiz — aqui: 3
Ancestrais de X	Todos os nós no caminho de X até a raiz — ancestrais de J: {F, C, A}
Descendentes de X	Todos os nós na subárvore enraizada em X, exceto X mesmo

Exercício 05 — Construção e Propriedades Estruturais da Árvore

Contexto: Dado o seguinte conjunto de valores a serem inseridos **em sequência** em uma Árvore Binária de Busca (ABB):

50, 30, 70, 20, 40, 60, 80, 10, 25, 45, 65

Parte A — Implementação da estrutura

Defina o nó da árvore e implemente as funções:

```
1  typedef struct No {  
2      int dado;  
3      struct No *esq;  
4      struct No *dir;  
5  } No;  
6  
7  No* inserir(No *raiz, int valor);  
8  int alturaNo(No *no);  
9  int profundidadeNo(No *raiz, int valor, int prof);  
10 int grauNo(No *no);
```

Parte B — Consultas estruturais

Após construir a ABB com os valores acima, calcule e imprima:

- A **altura** dos nós: 50, 30, 70, 20 e 10.
- A **profundidade** dos nós: 50, 30, 70, 45 e 10.
- O **grau** de cada nó da árvore (percurso em-ordem).
- A **altura total** da árvore.

Parte C — Verificação manual

Desenhe (como comentário ASCII art no código) a estrutura da ABB resultante após todas as inserções e **verifique manualmente** se os valores calculados pelas funções estão corretos para ao menos três nós.

Requisitos técnicos:

- `alturaNo` deve retornar `-1` para ponteiro nulo.
- `profundidadeNo` deve retornar `-1` se o valor não existir na árvore.
- Libere toda a memória alocada com `free` ao final (`liberarArvore`).

Exercício 06 — Ancestrais, Descendentes e Relatório Completo de um Nó

Contexto: Usando a mesma ABB do Exercício 05, implemente funções para listar os **ancestrais** e **descendentes** de qualquer nó, e produza um relatório estrutural completo.

Parte A — Funções de parentesco

```

1  /* Imprime todos os ancestrais do no com 'valor',
2     do pai ate a raiz (ordem crescente de profundidade)
3     */
4  void imprimirAncestral(No *raiz, int valor);
5
6  /* Imprime todos os descendentes do no com 'valor'
7     (todos os nos da subarvore, exceto o proprio no) */
8  void imprimirDescendentes(No *raiz, int valor);
9
10 /* Conta quantos descendentes o no possui */
11 int contarDescendentes(No *no);

```

Parte B — Relatório completo de nó

Implemente a função:

```

1 void relatorioNo(No *raiz, int valor);

```

que, dado um valor, imprime na tela um relatório no formato:

```

+-----+
| RELATORIO DO NO: 30          |
+-----+
| Profundidade   : 1           |
| Altura        : 2           |
| Grau          : 2           |
| Ancestrais     : 50          |
| Descendentes   : 20 10 25 40 45 |
| Qtd Descend.   : 5           |
+-----+

```

Parte C — Teste sistemático

Chame `relatorioNo` para os valores: **50** (raiz), **30** (nó intermediário), **10** (folha) e **70** (nó intermediário).

Requisitos técnicos:

- `imprimirAncestral` deve ser implementada de forma **recursiva**.
- `imprimirDescendentes` deve usar percurso **em-ordem**.
- Se o valor não existir na árvore, exiba mensagem de erro apropriada.
- Todo o programa deve compilar sem erros com `gcc -Wall -Wextra -o prog Exercicio06.c`.

Orientações para Entrega

Data de Entrega	26/05/2026
Local de Entrega	GitHub — pasta: ED-20261-Atividades-N2/ED-20261-Atividade-N2-3
Formato dos Arquivos	Arquivos de código (.c) ou relatórios/diagramas (.pdf)
Nomenclatura	Cada arquivo deve seguir o padrão: Exercicionn-x.K onde nn é o número do exercício (com dois dígitos), x é a Parte indicada por letra (A, B, C, ...) e K é a extensão do arquivo (.c para códigos ou .pdf para texto/diagramas). <i>Exemplo (Exercício 3):</i> • Parte A (Código): Exercicio03-A.c • Parte B (Complexidade): Exercicio03-B.pdf • Parte C (Profundidade): Exercicio03-C.c
Compilação	Todos os arquivos .c devem compilar sem erros com: <pre>gcc -Wall -Wextra -o prog Exercicionn-x.c</pre>

Importantíssimo: A primeira linha de **cada arquivo** (.c ou .pdf) deve conter obrigatoriamente o seu RA e o seu Nome Completo.

RA: 111111111111 - Nome do Aluno

Exemplo: RA: 1234567890123 - Joaquim José da Silva Xavier

Atenção: Entregas fora do prazo, com nomenclatura incorreta ou que não compilem poderão receber desconto na nota. Certifique-se de que o repositório está *público* e que todos os arquivos foram corretamente enviados (*commit + push*) antes do prazo.

“Um programa sem ponteiros é como um chef sem faca — você sobrevive, mas não cozinha de verdade.”

Atividade Avaliativa N2-3 · Estrutura de Dados · 2026.1